CHAPTER 1

BACK-OF-THE-ENVELOPE CALCULATIONS

1 Put Back-of-the-Envelope Numbers in Perspective 2
2 Examples of Resource Estimation . . 7

Learn how to estimate resource needs like servers, storage, and bandwidth using back-of-the-envelope calculations for System Design.

1 Put Back-of-the-Envelope Numbers in Perspective

Back-of-the-envelope calculations (BOTECs) involve swift, approximate, and simplified estimations or computations typically done on paper or, figuratively, on the back of an envelope. While these calculations are not intended to yield precise results, they function as a quick and preliminary evaluation of crucial parameters and the feasibility of a system.
For example, let's say we're in a city and want to estimate the population of a particular neighborhood. We could count the number of houses in a sample area, estimate the average number of people per household, and then extrapolate to the whole neighborhood. Similar calculations can be used to check the validity of census data for some neighborhoods.

1.1 BOTECs in system design

A modern system is a complex web of computational resources connected via a network. Different kinds of nodes, such as load balancers, web servers, application servers, caches, in-memory databases, and storage nodes, collectively serve the clients. Such a system might be architected in different ways, including a monolithic architecture, a modular monolith architecture, or a microservices architecture. Precisely considering such richness at the design level (especially in an interview) isn't advisable, and sometimes, it's impossible. BOTECs help us ignore the nitty-gritty details of the system (at least at the design level) and focus on more important aspects, such as finding the feasibility of the service in terms of computational resources.
Some examples where we often need BOTECs are the following estimations:

- The number of concurrent TCP connections a server can support
- The number of requests per second (RPS) a web, database, or cache server can handle
- The storage requirements of a service

Using BOTECs, we abstract away the messy details specific to different kinds of servers used in the actual system, the different access latencies of system components, different throughput rates, and the different types of requests. As we move forward, we'll first look into these different server types, access latencies, throughput numbers, and request types to know the reality of the systems and see how complex they are. Then , abstracting away these details, we'll learn to estimate the number of RPS a server can handle. Finally , we'll practice bandwidth, servers, and storage estimation examples.

Types of data center servers

Data centers don't have a single type of server. Enterprise solutions use commodity hardware to save costs and develop scalable solutions. Below , we discuss the types of servers that are commonly used within a data center to handle different workloads.

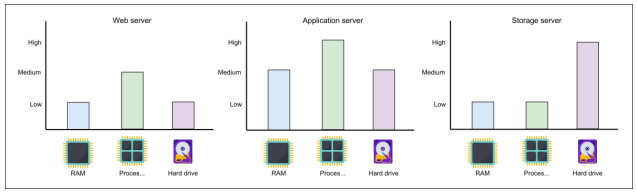

Figure 1.1: An approximation of the resources required on the web, application, and storage layer of the server, where the y-axis is a categorical axis with data points indicating levels of low, medium, and high resource requirements

Web servers For scalability, web servers are decoupled from application servers. **Web servers** are the first point of contact after load balancers. Data centers have racks full of web servers that usually handle API calls from clients. Depending on the service that's offered, the memory and storage resources in web servers can be small to medium. However, such servers require good processing resources. For example, Facebook has used a web server with 32 GB of RAM and 500 GB of storage space in the past.

Application servers **Application servers** run the core application software and business logic. The difference between web servers and application servers is somewhat fuzzy. Application servers primarily provide dynamic content, whereas web servers mostly serve static content to the client. They can require extensive computational and storage resources. Storage resources can be volatile and nonvolatile. Facebook has used application servers with a RAM of up to 256 GB and two types of storage—traditional rotating disks and flash—with a capacity of up to 6.5 TB.

Storage servers With the explosive growth of Internet users, the amount of data stored by giant services has multiplied. Additionally , various types of data are now being stored in different storage units. For instance, YouTube uses the following data stores:

1. **Blob storage:** This is used for its encoded videos.
2. **Temporary processing queue storage:** This can hold a few hundred hours of video content uploaded daily to YouTube for processing.
3. **Bigtable:** This is a specialized storage used for storing a large number of thumbnails of videos.
4. **Relational database management system (RDBMS):** This is for users' and videos' metadata (comments, likes, user channels, and so on).

Other data stores are still used for analytics, for example, Hadoop's HDFS. Storage servers mainly include structured (for example, SQL) and nonstructured (NoSQL) data management systems.
Returning to the example of Facebook, they've used servers with a storage capacity of up to 120 TB. With the number of servers in use, Facebook is able to house exabytes of storage. (One exabyte is 10^{18} bytes. By convention, we measure storage and network bandwidth in base 10, and not base 2.) However, the RAM of these servers is only 32 GB.

> **Note:** The servers described above aren't the only types of servers in a data center. Organizations also require servers for services like configuration, monitoring, load balancing, analytics, accounting, caching, and so on.

We need a reference point to ground our calculations. In the table below, we depict the capabilities of a typical serverAmazon EC2 M7i-flex instances, powered by 4th Generation Intel Xeon Scalable processors. that can be used in the data centers of today:

Typical Server Specifications

Component	Count
Processor	Intel Xeon (Sapphire Rapids 8488C)
Number of cores	64 cores
RAM	256 GB
Cache (L3)	112.5 MB
Storage capacity	16 TB

Table 1.1: Typical Server Specifications

Standard numbers to remember

A lot of effort goes into the planning and implementation of a service. But without any basic knowledge of the kinds of workloads machines can handle, this planning isn't possible. Latencies play an important role in deciding the amount of workload a machine can handle. The table below depicts some of the important numbers system designers should know in order to perform resource estimation.

Important Latencies

Component	Time (nanoseconds)
L1 cache reference	0.9
L2 cache reference	2.8
L3 cache reference	12.9
Main memory reference	100
Compress 1KB with Snzip	3,000 (3 microseconds)
Read 1 MB sequentially from memory	9,000 (9 microseconds)
Read 1 MB sequentially from SSD	200,000 (200 microseconds)
Round trip within same datacenter	500,000 (500 microseconds)
Read 1 MB sequentially from SSD with speed ~1GB/sec SSD	1,000,000 (1 milliseconds)
Disk seek	4,000,000 (4 milliseconds)
Read 1 MB sequentially from disk	2,000,000 (2 milliseconds)
Send packet SF NYC (round trip)	71,000,000 (71 milliseconds)

Table 1.2: Important Latencies

Remember the order of magnitudeI (if one amount is an order of magnitude larger than another, it is ten times larger than the other. If it is two orders of magnitude larger, it is a hundred times larger.) difference between different components and operations is more important than remembering the exact numbers. For example, we should know that doing IO-bound work (for example, reading 1 MB data sequentially from the SSD disk) is two orders of magnitude slower than CPU-bound work (for example, compressing 1 KB data as snzip). You might be wondering why the data sizes are different in the comparison!

As long as the data to compress is readily available to the processor from L1, L2, or L3 caches, the time to compress will be relatively consistent. The data up to the size of the L3 cache of the server (which is normally a few MBs—45 MBs for a typical server, as mentioned above) fits entirely within the cache, and therefore, compressing data up to this limit will take almost the same time. This is because the processor can quickly access the data from the cache without incurring the additional latency associated with fetching data from slower levels of memory or storage.

Apart from the latencies listed above, throughput numbers are measured as queries per second (QPS) that a typical single-server datastore can handle.

Important Rates

QPS handled by MySQL	1000
QPS handled by key-value store	10,000
QPS handled by cache server	100,000–1 M

Table 1.3: Important Rates

The numbers above are approximations and vary greatly depending on a number of reasons, like the type of query (pointpointQuery and rangerangeQuery), the specification of the machine, the design of the database, the indexing, the load on the server, and so on.

> **Note:** For real projects, initial designs use BOTECs similar to the ones we use in a system design interview. As the design goes through iterations for real products, we might use reference numbers from some synthetic workload that match our requests (for example, spec int for CPU

characterizations and TPC-CTransaction Processing Performance Council Benchmark C (TPC-C) is a benchmark to compare the performance of online transaction processing systems. for measuring database transactions per unit time). Initial prototypes are used to validate design-level assumptions. Later on, built-in monitoring of resources and demand is carefully analyzed to find any bottlenecks and for future capacity planning.

Quiz:
Question 1: With reference to the throughput numbers given above, what will be your reply if an interviewer says that they think that for a MySQL database, the average count of queries per second handled is 2000?
Answer: 2000 is in the same order of magnitude as 1000. If, for some use cases,

we need more precision, we might use a range such as, say, 250 queries for complex queries, 1750 for simpler queries, and 1000 on average.
Question 2: With reference to the table above, why does a key-value store serve an order of magnitude more queries per second as compared to a MySQL database?
Answer: A typical key-value store has a simpler API (‘put‘ and ‘get‘) compared to a relational database query that needs to go through query planning before query execution. On the same lines, in-memory caches have ‘read‘

and ‘write‘ operations, which are again simpler than a database query.

If such a cache is primarily used for reading, it can serve even more requests per second. Like the memory hierarchy inside a server, we need to keep the relative benefits of these systems (relational database, key-value store, and in-memory caches) in perspective. Doing so allows us to use some reasonable number for databases and then use the order of magnitude difference to develop related numbers for key-value stores and caches.

Request types

We'll see in the next section that while estimating the number of requests a server can handle, we don't get into the details of what kind of requests we're going to calculate for. But in reality, all requests are not the same. Workloads (clients' requests) can be broadly classified into three categories: CPU-bound, memory-bound, and IO-bound.

- **CPU-bound requests:** These primarily depend on the processor of a node. An example of a CPU-bound request is compressing 1 KB of data as snzip. From the table above, we see that this operation takes 3 microseconds.
- **Memory-bound requests:** These are primarily bottlenecked by the memory subsystem. An example is reading 1 MB of data sequentially from the RAM of a node. From the table above, we see that such an operation takes 9 microseconds (that's *three times* slower than a CPU-bound operation!).
- **IO-bound requests:** These are primarily bottlenecked by the IO subsystem (such as disks or the network). An example is reading 1 MB of data sequentially from a disk. From the table above, we see that such an operation takes 200 microseconds (a whopping *66 times* slower than CPU-bound operations!)

Similar to BOTECs, we can say that if a CPU-bound request takes X time units to complete some work on a node, then memory-bound workloads are an order of magnitude slower (10X), and IO-bound workloads are two orders of magnitude slower (100X) than the CPU-bound workload. We do such simplifications to make any further calculations easier.

1.2 Abstracting away the complexities of real system

Above, we've seen the complexities involved in real systems. You might have realized that considering all such complexities at the design level, especially in a limited time frame such as an interview, is impractical. BOTECs are a valuable tool for making quick, high-level estimates and decisions in the early stages of system design or when a rapid assessment is needed. So , moving forward, we'll learn to perform back-of-the-envelope calculations.

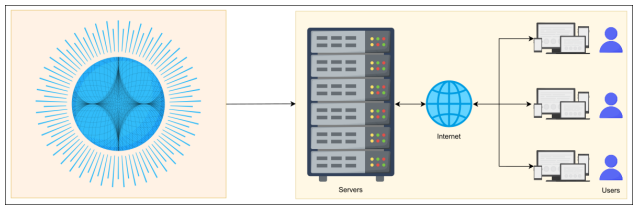

Figure 1.2: A real service is complex, where requests flow through many microservices, as shown on the left side of the image (which is an abstraction of the right side)

Request estimation in system design

This section discusses the number of requests a *typical server* can handle in a second. A real request will touch many nodes in a data center for different kinds of processing before a reply can be sent back to the client, and we'll accumulate all such work for our estimations.
The following equation calculates the CPU time to execute a program (request). For simplicity, we assume that each instruction can be executed in one clock cycle; therefore, CPI (clock cycles per instruction) is 1 in the following equation. Let 's assume the average clock rate for our servers' processor is 3.5 GHz (3,500,000,000 cycles per second). It 's reasonable to assume that a request will consume a few million instructions for full processing. For simplicity, let's assume that, on average, each request needs 3.5 million instructions.

$$\text{CPU}_{\text{time per program}} = \text{Instruction}_{\text{per program}} \times \text{CPI} \times \text{CPU}_{\text{time per clock cycle}}$$

Let's see that the units match on both sides of the equation. On the right side, we have the following:

- $\text{Instruction}_{\text{per program}}$: This is a count of the instructions the program (request) consists of, so it has no unit.
- CPI: This is a count of the clock cycles required to process one instruction, so it has no unit.
- $\text{CPU}_{\text{time per clock cycle}}$: This is the time the CPU takes to complete one clock cycle, measured in seconds.

From this, we can see that on the right side, we have the result in seconds, which is the time taken by the CPU per program (request).
Now, we'll put the assumed values in the equation above. But before that, we'll find the CPU time per clock cycle given the CPU frequency equalling 3.5 GHz.

$$\text{Clock cycles per second for a CPU with a 3.5 GHz clock rate} = 3.5 \times 10^9$$

$$\text{CPU}_{\text{time per clock cycle}} = \frac{1}{3.5 \times 10^9}$$

Putting all the values together, we get:

$$\text{CPU}_{\text{time per program}} = (3.5 \times 10^6) \times 1 \times \frac{1}{3.5 \times 10^9} = 0.001 \text{ second}$$

$$\text{Total requests a CPU executes in 1 second} = \frac{1}{10^{-3}} = 1000 \text{ requests}$$

$$\text{Total requests a 64 core server executes in 1 second} = 64000 \text{ requests}$$

Note that by changing the assumptions (for example, the number of instructions in a request), we'll get different final answers. In the absence of more information from these measurements, our estimates are reasonable.
Note how we avoided the complexities related to CPU, memory, or io-bound requests and system architecture.

Doing so is the hallmark of BOTECs.
In the next lesson, we'll use RPS numbers for server estimation with other resources, such as storage and network bandwidth.

2 Examples of Resource Estimation

2.1 Introduction

Now that we've laid the foundation for resource estimation, let's make use of the knowledge we gained in the previous lesson to estimate resources like servers, storage, and bandwidth. Below , we consider the Twitter service, make assumptions, and based on those assumptions, we make estimations. Let 's jump right in!

2.2 Number of servers required

Let's make the following assumptions about a Twitter-like service.
Assumptions:

- There are 500 million (M) daily active users (DAU).
- A single user makes 20 requests per day on average.
- We know that a single server (with 64 cores) can handle 64000 RPS.
- Total requests per second $\frac{500M \times 20}{86400} = 115K$ requests per second.
- For the number of servers: $\frac{Number\ of\ Requests/sec}{RPS\ of\ server} = \frac{115K}{64000} \approx 2$ servers to handle all the requests.

  Quiz:
  Question 1: Can you identify a hidden assumption in our calculations above?
  Answer: When we divided the daily user requests by the number of seconds in a day, we assumed that those requests were uniformly distributed over those seconds. This raises the question if the above assumption is reasonable? Rarely will a service get such a nice, uniform distribution of requests. Doing the above is a simple strategy; what we get can be considered a lower bound on the respective resource.

  Plausibility test: For all BOTECs, we need to judge if our numbers seem reasonable. For example, if the estimate we obtained was two servers for a large service with millons of DAUs, that number can be a lower bound but seems far from reality.

Peak capacity

Often, large services need to be ready for any flash crowds. We can get an estimate for the peak capacity. We assume that there's a specific second in the day when all the requests of all the users arrive at the service simultaneously. We use it to get the capacity estimation for a peak load. To do better, we'll need request and response distributions (A request-response distribution refers to the statistical pattern or model that describes the timing and frequency of requests made to a system or service and the responses provided by that system or service. For example, we can measure the number and types of incoming requests in 24 hours.), which might be available at the prototyping level (The prototyping level refers to the early stage of product development, where a basic version is created for testing and design validation before full development. It helps identify and address issues early in the process.). We might assume that requests follow a particular type of distribution, for example, the Poisson distributionPoisson2.
By using DAU as a proxy for peak load for a *specific second*, we've avoided difficulties finding the distributions of requests. Therefore, the DAU will then become the number of requests per second. So the number of servers at peak load can be calculated as follows:

$$\text{Servers needed at peak load} = \frac{\text{Number of requests/second}}{\text{RPS of server}} = \frac{10 \text{ Billion}}{64,000} = 157\text{K}$$

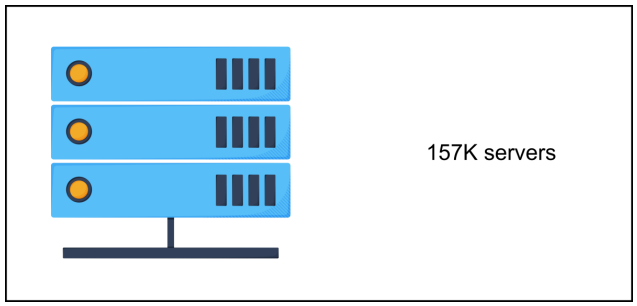

Figure 1.3: Number of servers required for a Twitter-like service

If our assumption is correct that all of the workloads can show up simultaneously in a specific second and each of our servers can handle 64,000 requests per second, we'll need the astronomical count of 157K servers! If that's not feasible, then we have two potential paths forward now, as explained below.

Improving the RPS of a server First, if we think our assumption for the peak load is correct, we can work out how many servers at max we can commission. Let 's assume we can employ 100,000 servers at most. That implies:

$$\frac{\text{Number of requests/second}}{\text{Max servers we can employ}} = \frac{10 \text{ Billion}}{100,000} = 100,000 \text{ Requests per second per server}$$

We'll need extensive engineering to bump the RPS we can extract from a server from 64,000 to 100,000! There are many examples where organizations relied on extensive engineering to improve the RPS of servers.
First example: WhatsApp reported in 2012 that it could manage 2 million concurrent TCP connections on one server. A report in 2017 mentioned that WhatsApp uses ~ 700 servers from IBM for its service. It's not clear what the specific specification of a server was.
Second example: A research system that was extensively optimized for IO won multiple sorting competitions in 2010. They sorted one trillion data records in 172 minutes using just a quarter of the computing resources of the other record holder, which means they improved the RPS three times more than the other record holder.
The examples above highlight that improving RPS for specific use cases is possible, though at the expense of focused R&D efforts and related dollar expenses.

Improving over the peak load assumption The second choice for us is to change our assumption for the peak load. Using the Pareto principle, also known as the 80/20 rule, estimating peak traffic can be a reasonable approach in many cases. The Pareto principle suggests that approximately 80% of the effects come from 20% of the causes. In the context of estimating peak traffic, we can make the assumption that 80% of our peak traffic occurs within 20% of the time (a 4.8-hour window in 24 hours).

$$\frac{\text{Number of requests/second}}{\text{RPS of server}} = \frac{\frac{0.8 \times 10 \text{ Billion}}{4.8 \times 60 \times 60 \text{ Seconds}}}{64,000} \approx 8 \text{ servers}$$

Once again, we've assumed that requests are equally distributed in the 4.8-hour window. The examples above show us that it makes a huge difference if the requests show up concurrently versus requests spread out over time. When systems are built on such assumptions, monitoring systems are in effect to make sure such assumptions are never violated. If the load gets higher than we predict, techniques like load-shedding,

circuit-breakers, and throttling can be employed. Dealing with an unexpected traffic peak is a difficult problem.

Quiz:
Question 1: Let's consider a service hosting the dynamic and personalized website of a large news organization. Due to some unexpected events, such as 9/11,

flash crowds are coming to the website to find out updates. It might be a situation where all the DAUs come in simultaneously. Such a situation will clearly break our usual load assumptions. Can you think of some way to gracefully degrade the service to meet such an unexpected load?
Answer: We can abandon per-user personalization for the time being because probably everyone cares about current events. Additionally, we can shift to a static-like website where content is pushed to CDN nodes and updated by the service when new updates come in. Doing so makes each request/response fast, and users get their data from the CDN, which will have multiple edge nodes near the customers. Service can also reduce the use of multimedia content or use it judiciously so the clients, whose networks might also be congested, could get the information in fewer bytes. The strategy suggested above will help the website go through the excessive load.

Cost of servers

We picked an EC2 instance type called m7i.16xlarge with a 64-core processor and 256 GB of RAM to get a handle on the dollar cost of servers. It 's powered by 4th-Generation Intel Xeon Scalable processors. The hourly cost of one such instance is $3.54816 with a 1-year contract plan.

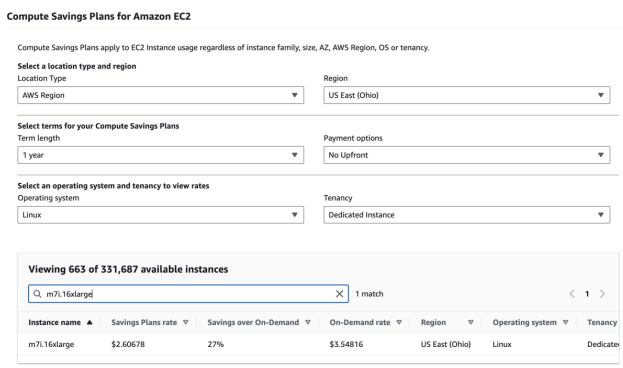

Figure 1.4: An approximate cost of an AWS instance

We've taken an EC2 instance from AWS with the following specifications:
EC2 Instance Specifications

Instance Size	vCPU	Memory (GiB)	Instance Storage (GB)	Network Bandwidth (Gbps)	EBS Bandwidth (Gbps)
m7i.16xlarge	64	256	EBS-Only	25	20

Table 1.4: EC2 Instance Specifications

The following table shows the cost of m7i.16xlarge instances for two, eight, and 157K servers. The cost can quickly pile up, as we can see for the peak load case. In real projects, the dollar budget available for specific items, such as servers, is a hard constraint that the engineering team needs to meet.
Cost of servers

2.3 Storage requirements

In this section, we attempt to understand how storage estimation is done by using Twitter as an example. We estimate the amount of storage space required by Twitter for new tweets in a year. Let 's make the following assumptions to begin with:

Low bound server cost per hour	Cost under 80–20 assumptions per hour	Peak load cost per hour
2*$3.548 = $7.096	8*$3.548 = $28.38	157K*$3.548 = $557,061

Table 1.5: Cost of servers

- We have a total of 500M daily active users.
- Each user posts three tweets in a day.
- Ten percent of the tweets contain images, whereas five percent of the tweets contain a video. Any tweet containing a video won't contain an image, and vice versa.
- An image is 200 KB and a video is 3 MB in size, on average.
- The tweet text and its metadata require a total of 250 bytesHistorically, one tweet was 140 characters, but the limit has been changed to 280 characters. We assume 250 bytes for simplicity. of storage in the database.

Then, the following storage space will be required per day:

- Total tweets: $500M \times 3 = 1500 \times 10^6$ tweets per day.
- Storage required for tweets in one day: $1500 \times 10^6 \times 250B = 375\ GB$.
- Storage required for 10 percent images for one day: $\frac{1500 \times 10^6 \times 10}{100} \times 200 \times 10^3 B = 30\ TB$.
- Storage required for five percent video content for one day: $\frac{1500 \times 10^6 \times 5}{100} \times 3 \times 10^6 B = 225\ TB$.
  - Total storage required for one day = 0.375TB + 30 TB + 225 TB $\approx$ 255 TB.
  - Storage required for one year = 365 $\times$ 255 TB = 93.08 PB.

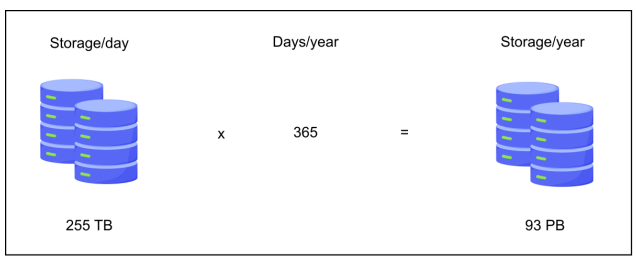

Figure 1.5: The total storage required by Twitter in a year

2.4 Bandwidth requirements

In order to estimate the bandwidth requirements for a service, we use the following steps:

1. Estimate the daily amount of incoming data to the service.
2. Estimate the daily amount of outgoing data from the service.
3. Estimate the bandwidth in Gbps (Gigabits per second) by dividing the incoming and outgoing data by the number of seconds in a day.

Incoming traffic: Let's continue from our previous example of Twitter, which requires 255 TBs of storage each day. Therefore, the incoming traffic should support the following bandwidth per second:

$$\frac{255 \times 10^{12}}{86400} \times 8 \approx 24 \text{ Gbps}$$

Note: We multiply by 8 in order to convert bytes (B) into bits (b) because bandwidth is measured in bits per second.

Outgoing traffic: Assume that a single user views 50 tweets in a day. Considering the same ratio of five percent and 10 percent for videos and images, respectively, for the 50 tweets, 2.5 tweets will contain video content, whereas five tweets will contain an image. Considering that there are 500M active daily users, we come to the following estimations:

- $500M \times 50\ tweets = 25$ billion tweets are viewed per day.
- Tweets viewed per second: $\frac{25*10^9}{86400} = 289 \times 10^3$ tweets per second.
- Bandwidth for tweets: $289 \times 10^3 \times 250 \times 8\ bits \approx 0.578\ Gbps$ (one tweet is equal to 250 bytes).
- Bandwidth of 10 percent images in tweets in a second: $289 \times 10^3 \times \frac{10}{100} \times 200 \times 10^3 \times 8\ bits = 46.24\ Gbps$.
- Bandwidth for five percent video tweets in a second: $289 \times 10^3 \times \frac{5}{100} \times 3 \times 10^6 \times 8\ bits = 346.8\ Gbps$.

The total outgoing traffic required will be equal to: $0.578 + 46.24 + 347 \approx 394\ Gbps$.

Twitter will need a total of 24 Gbps of incoming traffic and 393.62 Gbps of outgoing, assuming that the uploaded content is not compressed. Total bandwidth requirements $= 24 + 393.62 =$ 417.62 Gbps.

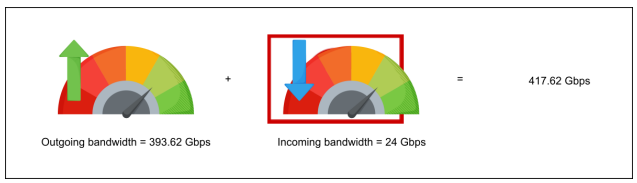

Figure 1.6: The total bandwidth required by Twitter

Once again, the calculations above depend on our specific assumption about the traffic mix (text versus images versus video) and read/write mix.

Quiz:
Question 1: We came up with the number of 93 PB for storage needs per year. Is this number plausible?
Answer: In 2023, 20 TB disks are readily available. That means we need about

5,000 disks to host 93 PB. Even if we incorporate three-way replication, we'll need just 15,000 disks. These many disks at a retail price of $400 will cost $6 million (though often organizations get robust volume discounts on such items).
Question 2: Isn't need for the network bandwidth of ~ 400 Gbps too high?
Answer: Many data centers of an organization are connected with high speed networks such as 1Tbps. Though our bandwidth needs towards the global

Internet might be high. But Twitter like organizations will have multiple, geographically dispersed data centers whose collective bandwidth can easily be matched what we calculated.

This lesson is a template for resource estimations in the rest of the course. We 'll use numbers from this lesson throughout the course. BOTECs enable us to show the system's feasibility under a specific design. During interviews, the ability to do such calculations exhibits a candidate's problem-solving skills in the face of the unknown. There 's only so much we can do in an interview. In the real world, however, organizations rely on real measurements for their specific workload and input to their back-of-the-envelope calculations.